openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

Securing the Modern Stack

Containers, Open Source, and the Tools Keeping Up

Aayan Mateen

Yogyakarta | October, 2026

About Me



I am **Aayan Mateen**
Founder and DevOps Consultant **Digital Aultis**
Leh, Ladakh, India

Aayan Mateen — how I got here

2024 Went to a CNCF meetup on DevSecOps and made supply chain security my problem.

2025 Handled a cryptominer incident on a Docker Compose stack. This talk is that story.

2025–26 Preparing for KCSA, learning trivy, grype, SBOMs and cosign one control at a time.

Now Running scanning, signed images and SBOMs on every merge in production.

What we'll cover

1. The incident: how a high-CPU alert turned into a cryptominer hunt.
2. Why it keeps happening: the perimeter moved, and attacks followed.
3. Know, Prove, Enforce: SBOMs, secrets, signing, hardening, SELinux.
4. What runs today: the free toolchain we use in production.

Questions welcome at the end.

It started with an alert

The alert	Alertmanager paged on high CPU: idle containers climbing all night.
The hunt	docker stats → btop → netstat → lsof found one rogue process.
The cause	CVE-2025-55182 “React2Shell”: unauthenticated RCE, CVSS 10.0.
The payload	A Monero miner running as root on plain Docker Compose.

Monitoring told us something broke. It couldn't tell us what was inside the image, or where it came from.

Finding the process, by hand

bash — vm-prod-02

```
# 1. which container is burning CPU?
$ docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}"
NAME        CPU %
web-app     398.72%
db          0.41%
# 2. which process inside it?
$ docker top web-app -o pid,user,pcpu,cmd
PID     USER    %CPU    CMD
4127    root    396     /tmp/.x/xmrig -o pool:3333
# 3. who is it talking to?
$ sudo netstat -tnp | grep 4127
tcp  0  0 10.0.4.17:41822   203.0.113.45:3333   ESTABLISHED 4127/xmrig
# 4. what is it holding open?
$ sudo lsof -p 4127 -a -i
```

No fancy tooling. Just the basics, used in order, on a plain VM.

Questions the alert couldn't answer

1. What is actually inside this image?
2. Where did every dependency come from?
3. What else, right now, is exposed the same way?

Monitoring told us something broke. It couldn't tell us what, or why.

Not edge cases. The new normal.

XZ Utils, 2024	A backdoor planted by a maintainer who spent two years earning trust. Found because SSH logins were half a second slow.
event-stream → registry campaigns	A popular npm package handed to a helpful volunteer who shipped a wallet stealer — the same pattern now shows up seeding container registries directly.
React2Shell, Dec 2025	CVE-2025-55182 — the one that hit us. Unauthenticated, CVSS 10.0, disclosed and weaponized in the same window.

Every one of these passed code review, or needed none to review.

So what replaces the perimeter?

1. Know — an accurate, machine-readable inventory of what is inside.

2. Prove — cryptographic evidence of who built it and how.

3. Enforce — a gate that refuses anything failing the first two.

Then run it on every build, forever. That last word is the hard part.

COMPOSITE SOFTWARE

Modern apps are assembled, not written

90%+

Open Source

The vast majority of modern codebases consist of third-party open-source components and frameworks.

Transitive

Hidden Supply Chain

Most dependencies are brought in implicitly by direct dependencies, creating deep nested vulnerability vectors.

Dynamic

Continuous Shift

Software composition updates continuously with upstream patches, builds, and automated dependency releases.

You aren't just shipping your code — you are shipping your entire upstream supply chain.

KNOW

What an SBOM actually is

It IS

- A machine-readable inventory of every component, version, and license in a build.
- The answer to "are we affected?" in minutes instead of days.

It IS NOT

- A vulnerability scanner or an automated fix.
- Proof that anything listed inside is safe.
- Useful if generated once and never looked at again.

Log4Shell cost most teams days just finding out where log4j was.

SBOMs: know what's inside

bash

```
# generate an SBOM from an image — syft or trivy, either works
$ syft registry:myapp:1.4.2 -o cyclonedx-json > sbom.cdx.json
$ trivy image --format cyclonedx -o sbom.json myapp:1.4.2

# scan the SBOM against vulnerability data
$ grype sbom:./sbom.cdx.json --fail-on high
$ osv-scanner --sbom=sbom.cdx.json
```

An SBOM answers “are we affected?” in minutes, not days. Store it with the release.

Scan it: what trivy tells you

bash — ci-runner

```
$ trivy image --exit-code 1 --severity HIGH,CRITICAL myapp:1.4.2
myapp:1.4.2 (node_modules)
Total: 1 (HIGH: 0, CRITICAL: 1)

LIBRARY                    VULNERABILITY    SEVERITY   INSTALLED  FIXED
react-server-dom-webpack   CVE-2025-55182   CRITICAL   19.1.0     19.1.2

$ echo $?
1
# non-zero exit: the pipeline stops, nothing ships

$ grype myapp:1.4.2 --fail-on high
ERROR discovered vulnerabilities at or above the severity threshold
```

Once a CVE is published, a nightly re-scan flags every image that carries it, including last quarter's.

Secret scanning: keep keys out of git

bash — ci-runner

```
# 1. scan the whole git history, not just the latest commit
$ gitleaks git -v .
Finding:  DB_PASSWORD=REDACTED
RuleID:   generic-api-key
File:     .env.production
Commit:   3f9a2c1
WRN leaks found: 1

# 2. only report secrets that still work
$ trufflehog git file://. --only-verified

# 3. and check the image too, layers remember everything
$ trivy image --scanners secret myapp:1.4.2
```

Once a secret is in git history, treat it as leaked. Rotate it first, then clean the history. Add gitleaks as a pre-commit hook.

Sign it, then make it checkable

bash

```
# sign - no long-lived key, identity comes from your OIDC provider
$ cosign sign myregistry/myapp:1.4.2

# verify - always pin BOTH the identity and the issuer
$ cosign verify myregistry/myapp:1.4.2 \
    --certificate-identity-regexp '^https://github.com/myorg/' \
    --certificate-oidc-issuer https://token.actions.githubusercontent.com
```

Signatures land in Rekor's public log. Add SLSA provenance to prove how it was built, not just what's inside.

Wolfi & Minimizing the Attack Surface

Wolfi OS (Chainguard)

- **Distroless by Design:** Contains zero shell, package manager, or unnecessary binaries.
- **Minimal Vulnerabilities:** Built specifically for modern container security with near-zero CVE exposure.

Least Attack Vector Base

- **Reduced Footprint:** Eliminates build tools and living-off-the-land attack paths.
- **Fine-grained Control:** Declarative image definitions with full SBOM compliance for every build.

By removing the shell and utilities entirely, attackers have no leverage even if they breach the application layer.

Harden every container

In the Dockerfile

- Pin base images by digest and use multi-stage builds.
- Set a real USER and a .dockerignore so secrets never ship.

In docker-compose

- user: "1000:1000", cap_drop: [ALL], no-new-privileges.
- read_only: true plus seccomp. Scan with trivy and grype on every build.

Our miner ran as root because nothing said it couldn't. These are text-file changes, not new tools.

BEFORE

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

The Dockerfile we shipped

Dockerfile

```
FROM node:20  ← floating tag, moves under you
WORKDIR /app
COPY . .  ← .env and .git go in too
RUN npm install  ← ignores the lockfile
RUN npm run build
EXPOSE 3000
CMD ["npm", "start"]  ← no USER, so it runs as root
```

Seven lines, four problems. Every one of them was in the image the miner landed in.

AFTER

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

Same app, hardened

Dockerfile

```
# stage 1: build tools live here and nowhere else
FROM node:20-alpine@sha256:<digest> AS build   ← pinned by digest
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci  ← exactly what's locked
COPY . .
RUN npm run build && npm prune --omit=dev

# stage 2: only the output ships
FROM node:20-alpine@sha256:<digest>
WORKDIR /app
COPY --from=build --chown=node:node /app ./
USER node  ← not root
CMD ["npm", "start"]
```

Pair it with a .dockerignore listing .env, .git and node_modules so secrets never reach a layer.

Least privilege in docker-compose

docker-compose.yml

```
services:
  web:
    image: myregistry/myapp:1.4.2@sha256:<digest>
    user: "1000:1000"  ← not root
    read_only: true  ← nowhere to drop a binary
    tmpfs: [/tmp]
    cap_drop: [ALL]  ← add back only what breaks
    security_opt:
      - no-new-privileges:true ← no setuid escalation
      - seccomp=./seccomp.json
    cpus: "1.0"  ← a miner can't eat every core
    mem_limit: 512m
```

None of this needs Kubernetes. It's a few lines per service in the file you already have.

AppArmor and SELinux in 60 seconds

AppArmor

- Path-based: a profile lists which files a program may touch.
- Easier to write. Docker applies docker-default to every container.

SELinux

- Label-based: every process and file has a type; policy decides.
- Finer-grained. Containers run as container_t and can't touch host files.

Even root inside a container hits this wall. New Tumbleweed and Leap 16 installs now default to SELinux.

Which one am I running?

bash — opensuse-host

```
# SELinux (Tumbleweed, Leap 16)
$ getenforce
Enforcing
$ ps -eZ | grep node
system_u:system_r:container_t:s0:c12,c87  2210 ?  node

# AppArmor (older Leap, Ubuntu, Debian)
$ sudo aa-status | head -2
apparmor module is loaded.
52 profiles are loaded.

# SELinux: relabel bind mounts so the container may read them
$ docker run -v ./data:/data:Z myapp:1.4.2
```

Don't turn it off to make an error go away. Read the denial: ausearch -m avc for SELinux, journalctl for AppArmor.

Check the guardrails actually hold

bash — vm-prod-02

```
# not root?
$ docker compose exec web id
uid=1000(node) gid=1000(node) groups=1000(node)

# read-only filesystem?
$ docker compose exec web touch /usr/bin/x
touch: /usr/bin/x: Read-only file system

# capabilities dropped?
$ docker compose exec web grep CapEff /proc/1/status
CapEff:  0000000000000000
```

Don't trust the compose file, check the running container. It takes thirty seconds.

Lock down your dependencies

Know the attacks

- Typosquatting: a package one keystroke from the one you meant.
- Dependency confusion: a public package shadowing your private name.

Defend early

- Enforce lockfiles: npm ci, pip install --require-hashes.
- Audit: npm audit, pip-audit, osv-scanner, Dependabot.

Scope internal packages and pin your registry. This wouldn't have stopped a zero-day, but it closes the more common mistake.

What runs today

- Prometheus + Alertmanager — the eyes. This is what caught it, and what we tuned afterward.
- docker stats, btop, netstat, lsof — still the runbook for the first five minutes of any alert.
- Semgrep + SonarQube — static analysis on our own source, every pull request.
- Trivy + Grype — image scanning on build and on a nightly schedule.
- Syft + Cosign — an SBOM and a signature on every release we ship.

None of this needed a budget. It needed a week, one uncomfortable meeting, and starting somewhere.

Thank You

Questions, war stories, and corrections all welcome.